

Flujo ARSO - Oposición

Estado actual

El flujo de **Oposición** fue implementado y probado siguiendo la misma estructura usada en:

- Acceso
- Rectificación
- Supresión

La oposición permite que el titular solicite que sus datos **no sigan siendo tratados para una finalidad específica**, sin eliminar físicamente sus datos.

¿Qué hace Oposición?

La oposición no borra datos.

A diferencia de Supresión:

Supresión = deja al titular inactivo operativamente.
Oposición = mantiene al titular activo, pero restringe el tratamiento.

Cuando una oposición es aprobada, en `organization-service` la persona queda:

```
isActive = true  
blocked = true  
deletionRequest = false  
anonymized = false  
dataStatus = PROCESSING_RESTRICTED
```

Flujo general

```
RECIBIDA  
↓  
IDENTIDAD_PENDIENTE  
↓  
IDENTIDAD_VERIFICADA  
↓  
EN_GESTION
```

↓
RESPONDIDA

o, si falla la identidad:

RECIBIDA
↓
IDENTIDAD_PENDIENTE
↓
IDENTIDAD_RECHAZADA
↓
RECHAZADA

Luego de una respuesta rechazada o respondida, queda disponible el plazo de reclamo ante Agencia.

Endpoints en arco-service

Base path:

/api/arso/opposition

1. Crear solicitud de oposición

POST /api/arso/opposition

Body:

```
{
  "arcoRequest": {
    "organizationId": "UUID_ORGANIZATION",
    "dataSubjectId": "UUID_PERSON",
    "assignedToUserId": null,
    "requestChannel": "WEB_PORTAL",
    "description": "Me opongo al uso de mis datos para marketing."
  },
  "cause": "DIRECT_MARKETING",
  "reason": "No deseo recibir comunicaciones comerciales.",
  "processingPurpose": "Marketing directo",
  "opposedTreatment": "Envío de publicidad personalizada"
}
```

Resultado esperado:

```
ArcoRequest.status = RECIBIDA
ArcoRequest.identityVerificationStatus = PENDIENTE
OppositionRequest.oppositionStatus = IDENTIDAD_PENDIENTE
```

2. Verificar identidad

```
PATCH /api/arso/opposition/{requestId}/verify-identity
```

Body aprobado:

```
{
  "verified": true,
  "comment": "Identidad verificada correctamente."
}
```

Resultado esperado:

```
ArcoRequest.status = EN_GESTION
ArcoRequest.identityVerificationStatus = VERIFICADA
OppositionRequest.oppositionStatus = EN_GESTION
```

Body rechazado:

```
{
  "verified": false,
  "comment": "No fue posible verificar la identidad del titular."
}
```

Resultado esperado:

```
ArcoRequest.status = RECHAZADA
ArcoRequest.identityVerificationStatus = RECHAZADA
OppositionRequest.oppositionStatus = IDENTIDAD_RECHAZADA
```

3. Responder solicitud

```
PATCH /api/arso/opposition/{requestId}/respond
```

Aprobación

```
{
  "approved": true,
  "observations": "Se acoge la oposición al tratamiento con fines de marketing.",
  "exceptionApplies": false
}
```

Resultado esperado:

```
ArcoRequest.status = RESPONDIDA
OppositionRequest.oppositionStatus = RESPONDIDA
OppositionRequest.decision = APPROVED
```

Además se llama a `organization-service` para aplicar la restricción operativa:

```
PATCH /api/organizations/{organizationId}/persons/{personId}/restrict-
processing
```

Rechazo

```
{
  "approved": false,
  "rejectionReason": "Existen fundamentos legítimos para continuar el tratamiento.",
  "overridingLegitimateGrounds": true
}
```

Resultado esperado:

```
ArcoRequest.status = RECHAZADA
OppositionRequest.oppositionStatus = RECHAZADA
OppositionRequest.decision = REJECTED
```

Causales implementadas

Enum:

OppositionCause

Causales MVP:

LEGITIMATE_INTEREST
DIRECT_MARKETING
PUBLIC_SOURCE

LEGITIMATE_INTEREST

Se usa cuando el titular se opone a un tratamiento basado en interés legítimo del responsable.

Ejemplo:

Me opongo a que mis datos sean usados para análisis interno basado en interés legítimo.

Rechazo por defecto:

Solicitud rechazada: existen motivos legítimos imperiosos para continuar el tratamiento.

DIRECT_MARKETING

Se usa cuando el titular se opone al tratamiento de datos con fines de marketing, publicidad o comunicaciones comerciales.

Ejemplo:

No deseo que mis datos sean usados para marketing directo.

Rechazo por defecto:

Solicitud rechazada: no se acreditó suficientemente la oposición al tratamiento de marketing.

PUBLIC_SOURCE

Se usa cuando el titular se opone al tratamiento de datos obtenidos desde una fuente pública.

Ejemplo:

```
Me opongo al uso de mis datos obtenidos desde una fuente pública.
```

Rechazo por defecto:

```
Solicitud rechazada: existe una base legal válida para continuar el  
tratamiento.
```

Causales de rechazo implementadas

El rechazo puede ser manual o automático por excepción.

Rechazo manual

Si el body incluye:

```
{  
  "approved": false,  
  "rejectionReason": "Motivo específico del rechazo."  
}
```

Se guarda ese motivo en:

```
OppositionRequest.rejectionReason  
OppositionRequest.responseSummary  
ArcoRequest.resolutionSummary
```

Rechazo por excepción legal

Si se envía:

```
{  
  "approved": true,
```

```
"exceptionApplies": true  
}
```

El sistema rechaza la solicitud igualmente.

Motivo aplicado:

Solicitud rechazada: existen fundamentos legales que permiten continuar el tratamiento.

Campos de rechazo contemplados

En `OppositionResponseDTO` se implementaron:

```
private Boolean overridingLegitimateGrounds;  
private Boolean legalObligationApplies;  
private Boolean publicInterestApplies;  
private Boolean exceptionApplies;
```

Estos campos permiten respaldar el rechazo cuando:

- Existen motivos legítimos imperiosos.
- Existe una obligación legal.
- Existe interés público.
- Aplica otra excepción legal.

Archivos principales en arco-service

Controller

```
com.example.demo.arco.oposicion.controller.OppositionController
```

Responsable de exponer:

```
POST /api/arso/opposition  
PATCH /api/arso/opposition/{requestId}/verify-identity  
PATCH /api/arso/opposition/{requestId}/respond
```

Service

```
com.example.demo.arco.oposicion.service.OppositionService
```

Responsable de:

```
crear solicitud  
verificar identidad  
aprobar oposición  
rechazar oposición  
aplicar restricción en organization-service  
enviar correos  
actualizar estados
```

Entity

```
com.example.demo.arco.oposicion.model.OppositionRequest
```

Campos principales:

```
id  
arcoRequest  
oppositionStatus  
cause  
decision  
reason  
processingPurpose  
opposedTreatment  
overridingLegitimateGrounds  
legalObligationApplies  
publicInterestApplies  
exceptionApplies  
rejectionReason  
responseSummary  
createdAt  
updatedAt
```

Repository

```
com.example.demo.arco.oposicion.repository.OppositionRequestRepository
```


Método principal:

```
Optional<OppositionRequest> findByArcoRequest_Id(UUID arcoRequestId);
```

DTOs

```
CreateOppositionDTO  
OppositionResponseDTO
```

CreateOppositionDTO

Recibe:

```
arcoRequest  
cause  
reason  
processingPurpose  
opposedTreatment
```

OppositionResponseDTO

Recibe:

```
approved  
observations  
rejectionReason  
overridingLegitimateGrounds  
legalObligationApplies  
publicInterestApplies  
exceptionApplies
```

Enums

```
OppositionCause  
OppositionDecision  
OppositionStatus
```

Archivos principales en organization-service

PersonService

Archivo:

```
cl.privdata.organizationService.service.PersonService
```

Se agregó el método:

```
restrictProcessing(UUID organizationId, UUID personId, String purpose)
```

Efecto:

```
blocked = true  
dataStatus = PROCESSING_RESTRICTED
```

Validaciones:

```
No permite restringir una persona anonimizada.  
No permite restringir una persona en proceso de supresión.
```

Por eso, si la persona está en:

```
DELETION_REQUESTED
```

la oposición será rechazada a nivel operativo.

DataStatus

Archivo:

```
cl.privdata.organizationService.enums.DataStatus
```

Debe contener:

```
ACTIVE,  
BLOCKED,  
DELETION_REQUESTED,
```

```
PROCESSING_RESTRICTED,  
ANONYMIZED
```

Controller de personas

Debe existir endpoint interno:

```
PATCH /api/organizations/{organizationId}/persons/{personId}/restrict-  
processing
```

Este endpoint es llamado desde `arco-service`.

OrganizationClient en arco-service

Archivo:

```
com.example.demo.client.OrganizationClient
```

Debe contener:

```
public void restrictProcessing(  
    UUID organizationId,  
    UUID personId,  
    String purpose  
) {  
    orgClient.patch()  
        .uri(organizationServiceUrl  
            + "/api/organizations/"  
            + organizationId  
            + "/persons/"  
            + personId  
            + "/restrict-processing")  
        .retrieve()  
        .toBodilessEntity();  
}
```

Correos

Se usan los mismos métodos de notificación que en los otros flujos ARSO:

```
notificarCreacion()  
notificarCambioEstado()  
notificarResolucion()
```

Los correos se visualizan en:

```
http://localhost:8025
```

Consideraciones importantes

1. Oposición no elimina datos

No se hace borrado físico ni lógico.

Solo se limita el tratamiento operativo.

2. No debe probarse con personas en DELETION_REQUESTED

Si una persona ya tiene supresión aprobada, queda:

```
dataStatus = DELETION_REQUESTED
```

En ese caso, la oposición falla porque el titular ya está bloqueado operativamente por supresión.

Para probar oposición se recomienda usar una persona nueva o activa.

3. La persona debe estar activa

Estado recomendado para pruebas:

```
isActive = true  
blocked = false  
deletionRequest = false  
anonymized = false  
dataStatus = ACTIVE
```

Flujo probado correctamente

1. Crear solicitud de oposición.
2. Verificar identidad.
3. Aprobar oposición.
4. Llamar a organization-service.
5. Aplicar restricción.
6. Confirmar que la persona queda en:

```
dataStatus = PROCESSING_RESTRICTED  
blocked = true  
isActive = true
```

Resumen corto

Oposición permite que el titular se oponga a un tratamiento específico de sus datos.

No elimina al titular.

Si se aprueba, organization-service marca la persona como PROCESSING_RESTRICTED y blocked = true.

Si se rechaza, se guarda la causal de rechazo y se habilita el plazo para reclamar ante Agencia.